

Capstone Project Documentation

Internal inventory and export operations platform for small craft producers

Project Summary

27 finalized REST endpoints

45 of 45 automated integration tests passed

Azure App Service + SQLite + Azure AI Foundry

DOCUMENT CONTROL

Version: 1.0 - Final

Prepared: August 24, 2026

Repository: github.com/RuhangLiu/SmartStock

Production: dyedwild-smartstock.azurewebsites.net

Audience: operators, support staff, developers, and course evaluators

Documentation Map

This package is the single submission artifact for SmartStock. It describes production support, installation, issue diagnosis, system usage, architecture, deployment, security, API coverage, and final test evidence. Source-level artifacts remain in the repository under docs/, server/tests/, and postman/.

Module 10 requirement	Where addressed
Production support, dependencies, monitoring, logs, tests	Chapter 1
System setup and deployment guide	Chapter 2
Issue description, diagnosis, research, resolution	Chapter 3
System usage and screenshots	Chapter 4
Architecture, request flow, database, environments	Chapter 5
Deployment pipeline and security	Chapter 6
API and testing traceability	Appendices A-B and companion XLSX

Table of Contents

Documentation Map

2

Table of Contents

3

Production Support and Testing

6

1. Purpose and Support Scope

6

2. Service Dependency Diagram

6

3. Component Monitoring and Health Checks

6

4. Logs and Operational Evidence

7

5. Common Incidents and Recovery

7

6. Automated Test Strategy and Results

8

7. Manual Test Results

9

8. Post-Deployment Smoke Test Results

9

9. Backup, Recovery, and Limitations

10

System Setup Guide

11

1. Purpose

11

2. Prerequisites

11

3. Obtain the Source

11

4. Install Dependencies

11

5. Local Configuration

11

6. Build and Run Locally

12

7. Database Initialization

12

8. Validate the Local Setup

13

9. Postman Setup

13

10. Azure App Service Deployment

14

11. Validate the Azure Deployment

15

12. Rollback

15

13. Setup Completion Checklist

15

Issue Diagnosis, Research, and Resolution

16

1. Troubleshooting Method

16

2. Issue Record: API Connection Refused

16

3. Issue Record: 401 Session Expired or Invalid

17

4. Issue Record: SQLite Persistence on Azure

18

5. Issue Record: Local Product Image Upload

19

6. Issue Record: Azure AI Model Access and Safe Fallback	20
7. Issue Record: Incorrect Explanation of Postman Request Flow	21
8. Knowledge Sharing	21

System Usage Guide

22

1. Audience and Purpose	22
2. Access Information	22
3. Sign In and Language	23
4. Dashboard	23
5. Products	23
6. Inventory Movement History	24
7. Sales	26
8. Orders	26
9. Customers	26
10. Reports and Settings	27
11. Database Viewer	27
12. AI Operations Briefing	27
13. Sign Out	28
14. Known Limitations and Gotchas	28
15. Support Contact	29

System Architecture

30

1. Architecture Overview	30
2. Component Responsibilities	30
3. Request Flow	31
4. API Layer	31
5. Data Layer	32
6. Hosting Environments	32
7. Deployment Architecture	32
8. Security Boundaries	33
9. Architecture Constraints	33

Deployment Pipeline and Security Considerations

34

1. Deployment Pipeline	34
2. Approval and Rollback	34
3. Authentication and Authorization	34
4. Database Security	35
5. File Upload Security	35
6. Azure AI Security	35

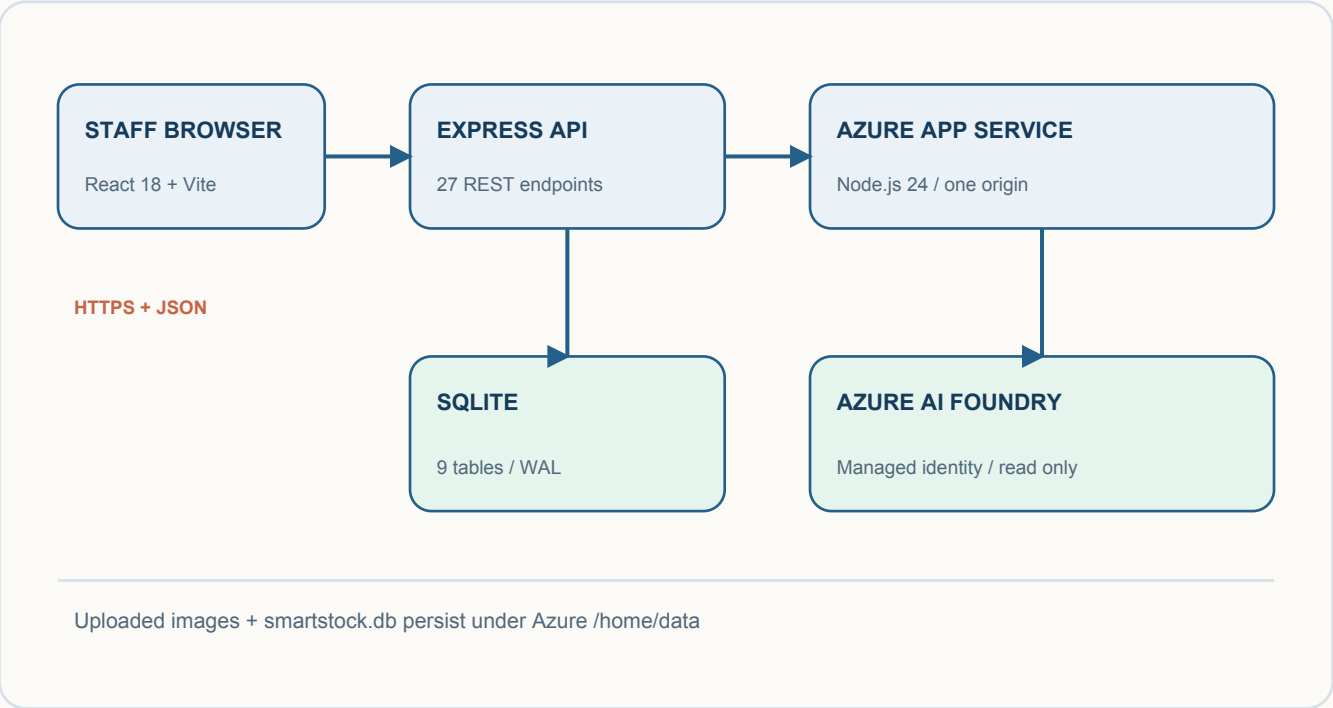
7. Secret Management	35
8. Production Hardening Recommendations	36
API Endpoint Inventory	37
Endpoint Count Reconciliation	37
Test and Smoke Evidence	39
Automated Test Summary	39
Coverage Categories	39
Production Smoke Evidence	39
Evidence Handling	40
Appendix C - Artifact Index	41
Final Evidence Statement	41

Production Support and Testing

1. Purpose and Support Scope

This document explains how to monitor, validate, troubleshoot, and recover the SmartStock production application. It covers the React frontend, Express REST API, SQLite database, product-image storage, Azure App Service, Azure AI integration, and GitHub deployment path.

2. Service Dependency Diagram



SmartStock system architecture

The browser sends HTTPS requests to the React interface and the Express REST API hosted in the same Azure App Service. Express authenticates each protected request, enforces role permissions, validates input, and reads or writes SQLite. The AI controller reads only approved aggregates and contacts Azure AI through the App Service managed identity. GitHub supplies the source revision deployed by Azure Deployment Center.

3. Component Monitoring and Health Checks

SmartStock does not expose a dedicated public `/health` endpoint. Support personnel use the following least-privilege smoke sequence instead.

Component	Health signal	Healthy result	Where to check
Azure App Service	Landing page request	Login page loads over HTTPS	Production URL and Azure Overview
React frontend	Static assets and navigation	Login form renders without a blank screen	Browser and browser console

Component	Health signal	Healthy result	Where to check
Express API	Authentication request	Valid demo login returns HTTP 200	Postman or application login
Authentication	Current-user request	<code>GET /api/auth/me</code> returns the signed-in user	Postman or workspace header
SQLite	Approved table catalog	Database Viewer shows 9 tables	Administrator Database page
Inventory data	Product listing	<code>GET /api/products</code> returns product records	Products page or Postman
Uploaded images	Static file request	Image returns HTTP 200	Product card and <code>/uploads/products/...</code> URL
Azure AI	Briefing generation	Dashboard shows AZURE AI , four insights, and Read only	Administrator dashboard
AI audit log	Activity row	<code>ai_activity_logs</code> receives a success or error event	Database Viewer
Deployment	Build and release status	Latest main-branch deployment succeeds	Azure Deployment Center

4. Logs and Operational Evidence

Evidence	Content	Access path
Application console	Startup message, unhandled errors, AI fallback warnings	Local terminal or Azure Log Stream
Azure Log Stream	Runtime stdout/stderr when App Service logging is enabled	Azure Portal > App Service > Monitoring > Log stream
Deployment logs	Source commit, build output, dependency installation, startup result	Azure Portal > App Service > Deployment Center > Logs
Persistent log folder	App Service filesystem logs	<code>/home/LogFiles</code> through SSH or Advanced Tools
AI audit table	Provider, status, language, row count, duration, user, timestamp	Database Viewer > Ai Activity Logs
Automated test JSON	Test ID, scenario, expected value, actual value, status, duration	<code>evidence/API_Test_Results.json</code>

Passwords, session tokens, request authorization headers, and Azure access tokens must not be copied into tickets or screenshots.

5. Common Incidents and Recovery

5.1 Browser or Postman reports ECONNREFUSED

- 1 Confirm the requested base URL and port.
- 2 For local work, start SmartStock from the repository root with `npm start`.
- 3 Confirm the terminal displays `SmartStock backend listening on http://localhost:4000`.
- 4 Set Postman `baseUrl` to `http://localhost:4000` for local testing or the production HTTPS URL for Azure.
- 5 Retry login before testing protected endpoints.

5.2 API returns 401 Session expired or invalid

- 1 Send `POST /api/auth/login` again.

- 2 Replace the Postman bearer token with the newly returned token.
- 3 Confirm the `Authorization` header is `Bearer <token>`.
- 4 Do not reuse a token after logout. Sessions expire after seven days.

5.3 SQLite data disappears after deployment or restart

- 1 Confirm `SMARTSTOCK_DB_PATH=/home/data/smartstock.db`.
- 2 Confirm `WEBSITES_ENABLE_APP_SERVICE_STORAGE=true`.
- 3 Confirm the app is using one App Service instance; this SQLite design is not intended for multi-instance writes.
- 4 Use Azure SSH to confirm the database and WAL files exist under `/home/data`.
- 5 Restart the application only after verifying the path. Do not replace the database with a stale local copy.

5.4 Product image upload fails

- 1 Confirm the user is an administrator.
- 2 Confirm the file is JPG, PNG, or WebP and no larger than 5 MB.
- 3 Confirm `SMARTSTOCK_UPLOAD_DIR=/home/data/uploads/products` in Azure.
- 4 Confirm `/home` persistence is enabled and the folder is writable.
- 5 Retry with a small known-good PNG and check the returned `image_url`.

5.5 Azure AI is unavailable

- 1 Confirm `AZURE_OPENAI_ENABLED=true`, the endpoint, and the deployment name.
- 2 Confirm the App Service system-assigned identity is enabled.
- 3 Confirm that identity has the `Cognitive Services OpenAI User` role on the AI resource.
- 4 Review the application warning and the latest `ai_activity_logs` record.
- 5 SmartStock automatically returns a deterministic local read-only briefing, so inventory operations remain available while AI access is repaired.

5.6 Azure deployment fails or the app does not start

- 1 Open Deployment Center and identify the failing commit and build step.
- 2 Confirm Node.js 24 is selected and the repository root contains `package.json`.
- 3 Confirm the frontend build completed and `client/dist/index.html` exists.
- 4 Confirm `npm start` launches `server/index.js`.
- 5 If the new deployment is unusable, redeploy the last known-good commit and repeat the smoke test.

6. Automated Test Strategy and Results

The automated suite starts the real Express application on an ephemeral local port and uses an isolated temporary SQLite database. It sends real HTTP requests and compares status codes, response bodies, permissions, validation behavior, and data changes with expected results. The test database and uploaded test files are removed after execution.

Test layer	SmartStock implementation	Result
Validation/controller behavior	Invalid credentials, duplicate SKU, invalid image type, negative stock, and authorization cases	Covered within HTTP suite
API integration	Express routes, authentication middleware, controllers, SQLite, file upload, and transactions	45/45 passed
Workflow/end-to-end API	Login-to-operation flows, sale-to-stock verification, adjustment-to-history verification, logout invalidation	Passed
Manual UI validation	Login, navigation, dashboard, products, inventory, database viewer, and AI briefing	Passed locally
Post-deployment smoke validation	Azure landing page, login, dashboard, database viewer, and Azure AI briefing	Passed August 24, 2026

Command:

```
cd server
npm test
```

Expected result:

```
SmartStock API tests: 45/45 passed
```

Detailed evidence is available in [API Test Cases and Results](API_Test_Cases_and_Results.xlsx) and [the machine-readable JSON report](../evidence/API_Test_Results.json).

7. Manual Test Results

ID	Scenario	Expected	Actual	Result
MT-01	Sign in as administrator	Dashboard opens with admin-only features	Dashboard, AI, and Database navigation displayed	Pass
MT-02	Switch interface to Chinese and back	Labels translate without moving core navigation	Language switched and layout remained usable	Pass
MT-03	Open product catalog	Eight seeded products and images display	Eight products displayed with images	Pass
MT-04	Open inventory history	Movement rows and filters load	Initial movement history loaded	Pass
MT-05	Open Database Viewer	Approved tables load without edit controls	Nine approved tables displayed read-only	Pass
MT-06	Generate local AI preview	Four read-only insights display	Four insights displayed; no mutation controls	Pass

8. Post-Deployment Smoke Test Results

Smoke test date: **August 24, 2026** Target: <https://dyedwild-smartstock.azurewebsites.net>

Check	Observed result	Status
HTTPS landing page	SmartStock sign-in page loaded	Pass
Administrator login	Dashboard opened with Alicia Chen / Admin	Pass

Check	Observed result	Status
Dashboard data	Product, inventory, revenue, order, and customer metrics loaded	Pass
SQLite integration	Database Viewer returned the approved table catalog	Pass
Azure AI	AZURE AI briefing returned four read-only insights	Pass
Data safety	Smoke test used read-only navigation and briefing generation only	Pass

9. Backup, Recovery, and Limitations

- SQLite and product uploads are stored under Azure `/home/data` for persistence.
- Before any database replacement, stop writes and copy the database together with active WAL/SHM files or create a consistent SQLite backup.
- The Free F1 demonstration plan may cold-start and is not an availability guarantee.
- SQLite is appropriate for this capstone and single-instance workload, not high-volume multi-instance production scaling.
- A formal alerting system and dedicated health endpoint are future enhancements.

System Setup Guide

1. Purpose

This guide allows a new developer to install, run, test, and deploy SmartStock from a clean environment. Commands assume macOS, Linux, or a terminal with equivalent Git and Node.js support.

2. Prerequisites

Requirement	Version or setting	Validation command
Git	Current supported release	<code>git --version</code>
Node.js	24.x	<code>node --version</code>
npm	Bundled with Node.js 24	<code>npm --version</code>
Browser	Current Chrome, Edge, Safari, or Firefox	Open a local URL
Azure account	Required only for cloud deployment	Sign in to Azure Portal
Postman	Optional for manual API tests	Import the supplied collection

SmartStock uses the built-in `node:sqlite` module. Node.js 24 is required; no separate SQLite npm driver is installed.

3. Obtain the Source

```
git clone https://github.com/RuhangLiu/SmartStock.git
cd SmartStock
```

Never commit `.env`, access tokens, production database copies, or real customer data.

4. Install Dependencies

Install root and workspace dependencies:

```
npm ci
npm ci --prefix client
npm ci --prefix server
```

The root application supplies the Azure entry point. `client` contains React/Vite. `server` contains Express, routes, SQLite integration, and tests.

5. Local Configuration

The application runs without a `.env` file by using safe local defaults.

Variable	Local default	Purpose
<code>PORT</code>	<code>4000</code>	Express listening port

Variable	Local default	Purpose
SMARTSTOCK_DB_PATH	server/database/smartstock.db	SQLite file location
SMARTSTOCK_UPLOAD_DIR	server/uploads/products	Uploaded product images
AZURE_OPENAI_ENABLED	false	Uses local read-only AI preview when false
AZURE_OPENAI_ENDPOINT	Empty	Azure AI resource endpoint
AZURE_OPENAI_DEPLOYMENT	smartstock-ai-mini	Azure model deployment name
AZURE_OPENAI_TIMEOUT_MS	12000	AI request timeout in milliseconds

To override a value for one terminal session:

```
PORT=4000 SMARTSTOCK_DB_PATH=./server/database/smartstock.db npm start
```

6. Build and Run Locally

6.1 Production-style local run

```
npm run build
npm start
```

Open <http://localhost:4000>.

6.2 Frontend development mode

In terminal 1:

```
cd server
npm start
```

In terminal 2:

```
cd client
npm run dev
```

Open the URL printed by Vite. The frontend development configuration forwards API requests to the Express backend.

7. Database Initialization

On first start, `server/src/models/db.js`:

- 1 Creates the database folder and SQLite file.
- 2 Enables foreign keys and WAL journal mode.
- 3 Creates nine application tables.
- 4 Adds missing product columns for backward compatibility.
- 5 Creates indexes.

- 6 Seeds administrator, employee, product, order, customer, and settings demonstration data only when the relevant tables are empty.
- 7 Creates initial inventory movement records.

Local database path:

```
server/database/smartstock.db
```

Do not commit runtime `smartstock.db`, `smartstock.db-wal`, or `smartstock.db-shm` files.

8. Validate the Local Setup

8.1 Browser validation

- 1 Open `http://localhost:4000`.
- 2 Sign in with the administrator demonstration account.
- 3 Confirm Dashboard metrics load.
- 4 Open Products and confirm eight seeded product images display.
- 5 Open Database and confirm nine approved tables display.
- 6 Generate the AI briefing. Local development should show a read-only local preview unless Azure variables and managed identity are available.

8.2 Automated validation

```
cd server  
npm test
```

Expected: `45/45 passed`. The test runner uses a temporary SQLite database and does not modify local demonstration data.

8.3 API validation

The login request must go to the **backend API endpoint**, not to a UI page:

```
POST http://localhost:4000/api/auth/login  
Content-Type: application/json  
  
{  
  "email": "admin@smartstock.com",  
  "password": "admin123"  
}
```

Use the returned token as `Authorization: Bearer <token>` for protected requests.

9. Postman Setup

- 1 Import `docs/SmartStock_API.postman_collection.json`.
- 2 Set collection variable `baseUrl`:
 - Local: `http://localhost:4000`

- Azure: `https://dyedwild-smartstock.azurewebsites.net`
- 1 Run **POST** **Log in** first.
- 2 Store the returned bearer token in the collection variable used by protected requests.
- 3 Use a new unique SKU for each create-product demonstration.

10. Azure App Service Deployment

10.1 Create or select resources

- Resource group: `smartstock-rg`
- Linux Web App: `dyedwild-smartstock`
- Runtime: Node.js 24
- App Service plan: Free F1 for the capstone demonstration
- Source: GitHub repository main branch through Deployment Center

10.2 Application settings

Configure the following under App Service > Settings > Environment variables:

```
NODE_ENV=production
SMARTSTOCK_DB_PATH=/home/data/smartstock.db
SMARTSTOCK_UPLOAD_DIR=/home/data/uploads/products
WEBSITES_ENABLE_APP_SERVICE_STORAGE=true
SCM_DO_BUILD_DURING_DEPLOYMENT=true
AZURE_OPENAI_ENABLED=true
AZURE_OPENAI_ENDPOINT=https://dyedwild-smartstock-ai.openai.azure.com
AZURE_OPENAI_DEPLOYMENT=smartstock-ai-mini
AZURE_OPENAI_TIMEOUT_MS=12000
```

Do not add an Azure AI API key. Production uses managed identity.

10.3 Enable Azure AI identity access

- 1 Open App Service > Identity.
- 2 Enable the system-assigned managed identity.
- 3 Open the Azure AI resource > Access control (IAM).
- 4 Add the **Cognitive Services OpenAI User** role.
- 5 Select the `dyedwild-smartstock` managed identity as the member.
- 6 Confirm the model deployment name matches `AZURE_OPENAI_DEPLOYMENT`.

10.4 Configure GitHub deployment

- 1 Open App Service > Deployment Center.
- 2 Select GitHub as the source.
- 3 Select the repository and `main` branch.

- 4 Save the deployment configuration.
- 5 Review the first build log.

The root build installs frontend dependencies, builds `client/dist`, and starts Express with `server/index.js`. Express serves both the compiled React interface and `/api` routes from one origin.

11. Validate the Azure Deployment

- 1 Open the production HTTPS URL.
- 2 Sign in as administrator.
- 3 Confirm Dashboard data loads from SQLite.
- 4 Open Database Viewer and confirm nine tables.
- 5 Generate the AI briefing and confirm the `AZURE AI` label.
- 6 Upload a small test image only when a mutation test is specifically required.
- 7 Review Deployment Center and Log Stream for errors.

12. Rollback

- 1 Open Deployment Center > deployment history.
- 2 Identify the last known-good main-branch deployment.
- 3 Redeploy that revision using the Azure-supported redeploy action or revert the faulty commit through a reviewed GitHub pull request.
- 4 Do not overwrite `/home/data/smartstock.db` during an application rollback.
- 5 Repeat the full post-deployment smoke test.

13. Setup Completion Checklist

- ☐ Node.js 24 and npm installed
- ☐ Repository cloned
- ☐ Dependencies installed
- ☐ Frontend build succeeds
- ☐ Express starts on the expected port
- ☐ Login page and dashboard load
- ☐ Database contains nine tables
- ☐ Automated test suite reports 45/45 passed
- ☐ Azure `/home/data` persistence configured
- ☐ Managed identity and Azure AI role configured
- ☐ Production smoke test completed

Issue Diagnosis, Research, and Resolution

1. Troubleshooting Method

For each incident, the team used the same sequence:

- 1 Record expected and actual behavior.
- 2 Reproduce the issue in a controlled environment.
- 3 Identify the failing layer: browser, React, Express, authentication, SQLite, Azure storage, or Azure AI.
- 4 Inspect the nearest available evidence: response body, terminal output, test report, Deployment Center, Log Stream, or audit table.
- 5 Compare the implementation with official documentation.
- 6 Apply the smallest scoped correction.
- 7 Re-run the specific scenario and the complete automated suite.

AI assistance was used to organize hypotheses and documentation, but conclusions were verified against the application code, HTTP results, SQLite state, Azure configuration, and official product documentation.

2. Issue Record: API Connection Refused

Description

- **Expected:** Postman sends a request to SmartStock and receives JSON.
- **Actual:** Postman displayed `ECONNREFUSED`, or the browser displayed `ERR_CONNECTION_REFUSED`.

Environment

Local macOS development, Express expected on port 4000, Postman collection using `{{baseUrl}}`.

Reproduction

- 1 Stop the Express process.
- 2 Send `POST {{baseUrl}}/api/auth/login`.
- 3 Observe that no process accepts the connection.

Diagnosis and Root Cause

The failure occurred before HTTP reached Express. Either the backend was not running or `baseUrl` referenced the wrong host or port. This is different from an HTTP 4xx/5xx response because no server response exists.

Research and Resources

- Express startup behavior and local terminal output.

- Postman request URL and collection variable inspection.
- Browser and operating-system connection error behavior.

Resolution

- 1 Run `npm start` from the repository root.
- 2 Confirm `SmartStock` backend listening on `http://localhost:4000`.
- 3 Set local `baseUrl` to `http://localhost:4000`.
- 4 Set Azure `baseUrl` to `https://dyedwild-smartstock.azurewebsites.net`.

Verification

The login endpoint returned HTTP 200 and a token. The automated test runner also opened an ephemeral HTTP port successfully when permitted by the local environment.

3. Issue Record: 401 Session Expired or Invalid

Description

- **Expected:** An authenticated administrator creates or updates a business record.
- **Actual:** The API returned HTTP 401 with `Session expired or invalid`.

Environment

Postman request with a previously saved bearer token.

Reproduction

- 1 Log in and save a token.
- 2 Log out or wait until the session expires.
- 3 Reuse the old token on a protected endpoint.

Diagnosis and Root Cause

`requireAuth` joins the submitted bearer token to the `sessions` and `users` tables and requires `expires_at` to be in the future. A logged-out, missing, malformed, or expired token fails this query.

Research and Resources

- `server/src/middleware/auth.js`
- HTTP Authorization header format
- Postman collection variables and headers

Resolution

- 1 Send a fresh login request.

- 2 Replace the collection bearer token.
- 3 Confirm the header contains exactly `Bearer <token>`.

Verification

The protected request succeeded with the new token. TC-45 also verified that logout invalidates the previous token.

4. Issue Record: SQLite Persistence on Azure

Description

- **Expected:** Products, sales, orders, customers, inventory movements, and uploaded images remain after an App Service restart.
- **Actual:** Data stored in a nonpersistent application folder could be lost during redeployment or restart.

Environment

Azure App Service on Linux with file-backed SQLite.

Reproduction

- 1 Configure the database inside a transient deployment directory.
- 2 Write demonstration data.
- 3 redeploy or replace the application container.
- 4 Observe that transient files are not a reliable persistence layer.

Diagnosis and Root Cause

App Service application content and transient container storage are not equivalent to durable application data. SmartStock needed an explicit path under Azure `/home` and persistent App Service storage enabled.

Research and Resources

- [Azure App Service Linux storage FAQ](#)
- [Azure custom container and `/home` persistence](#)
- [Node.js `node:sqlite` documentation](#)

Resolution

```
SMARTSTOCK_DB_PATH=/home/data/smartstock.db
SMARTSTOCK_UPLOAD_DIR=/home/data/uploads/products
WEBSITES_ENABLE_APP_SERVICE_STORAGE=true
```

The application creates required folders and uses WAL mode. Azure remains limited to one App Service instance because this capstone SQLite design is not intended for concurrent multi-instance writes.

Verification

Azure SSH showed `smartstock.db`, `smartstock.db-wal`, and `smartstock.db-shm` under `/home/data`. Production data remained available after deployment and the Database Viewer returned nine tables.

5. Issue Record: Local Product Image Upload

Description

- **Expected:** An administrator chooses a local image and the product record displays it.
- **Actual:** The original form accepted only an external image URL.

Environment

React product form, Express API, Azure App Service persistent storage.

Reproduction

- 1 Open Add Product.
- 2 Observe that only `Product Image URL` is available.
- 3 Attempt to reference a local file path, which a remote browser cannot serve as a public image.

Diagnosis and Root Cause

A local filesystem path is not a server-hosted URL. The application needed a multipart upload endpoint, server-side validation, persistent file storage, and a returned public application path.

Research and Resources

- Multer multipart upload behavior.
- Azure `/home` persistence documentation.
- Browser file-input and MIME type behavior.

Resolution

- 1 Added `POST /api/products/image`.
- 2 Restricted upload to administrators.
- 3 Accepted JPG, PNG, and WebP up to 5 MB.
- 4 Generated a UUID filename.
- 5 Stored files under the configured upload directory.
- 6 Returned `/uploads/products/<filename>` for product records.
- 7 Added file selection and image preview to React.

Verification

TC-31 confirmed employee denial, TC-32 confirmed unsupported-type rejection, and TC-33 confirmed a valid file was persisted and publicly addressable.

6. Issue Record: Azure AI Model Access and Safe Fallback

Description

- **Expected:** An administrator receives four current operations insights without exposing secrets or allowing database changes.
- **Actual:** Azure AI could be unavailable because of an incorrect endpoint, deployment name, managed identity, regional model access, or role assignment.

Environment

Azure App Service connected to an Azure AI Foundry resource.

Reproduction

- 1 Enable Azure AI with an invalid deployment or without identity permission.
- 2 Generate a briefing.
- 3 Observe a managed-identity or Azure AI request warning in application logs.

Diagnosis and Root Cause

Azure AI access requires both a valid model deployment and authorization for the App Service identity. Storing an API key in React would expose the credential, so authentication had to remain server-side.

Research and Resources

- [Microsoft guidance for App Service and Azure OpenAI with managed identity](#)
- [Microsoft Foundry managed identity guidance](#)
- Application controller and AI activity audit records

Resolution

- 1 Enabled App Service system-assigned identity.
- 2 Assigned **Cognitive Services OpenAI User** on the AI resource.
- 3 Stored only endpoint, deployment name, enable flag, and timeout in App Service settings.
- 4 Allowed only predefined aggregate inventory, order, and sales data.
- 5 Limited detailed rows and output length.
- 6 Parsed Azure output into a strict four-insight JSON structure.
- 7 Added a deterministic local read-only fallback.
- 8 Logged provider, status, duration, row count, language, and user without prompts or tokens.

Verification

Production displayed the **AZURE AI** badge and four read-only insights on August 24, 2026. TC-41 through TC-44 verified role restrictions, English and Chinese local previews, and safe audit logging.

7. Issue Record: Incorrect Explanation of Postman Request Flow

Description

- **Expected explanation:** Postman sends HTTP requests directly to Express REST API endpoints.
- **Incorrect explanation:** Postman sends requests to the web UI.

Diagnosis

The UI and API share one Azure host, but they are separate application layers. React is a client of the API. Postman is another client of the same API and does not need to operate through React.

Correct Flow

```
Postman -> HTTPS -> Express /api endpoint -> authentication/validation -> SQLite -> JSON response
```

Verification

Postman requests use URLs such as **/api/auth/login** and **/api/products**. The Express route definitions and returned JSON confirm the request reaches the backend API layer.

8. Knowledge Sharing

Resolved issues are shared through:

- this troubleshooting document;
- GitHub pull requests with scoped commit history;
- machine-readable test evidence;
- Postman collection examples;
- architecture and deployment diagrams;
- presentation and demonstration notes.

System Usage Guide

1. Audience and Purpose

This guide is for store administrators and employees. SmartStock is a staff-facing operations system, not a consumer storefront. It helps a small cross-border tie-dye business manage product information, inventory, sales, orders, and customer records in one workspace.

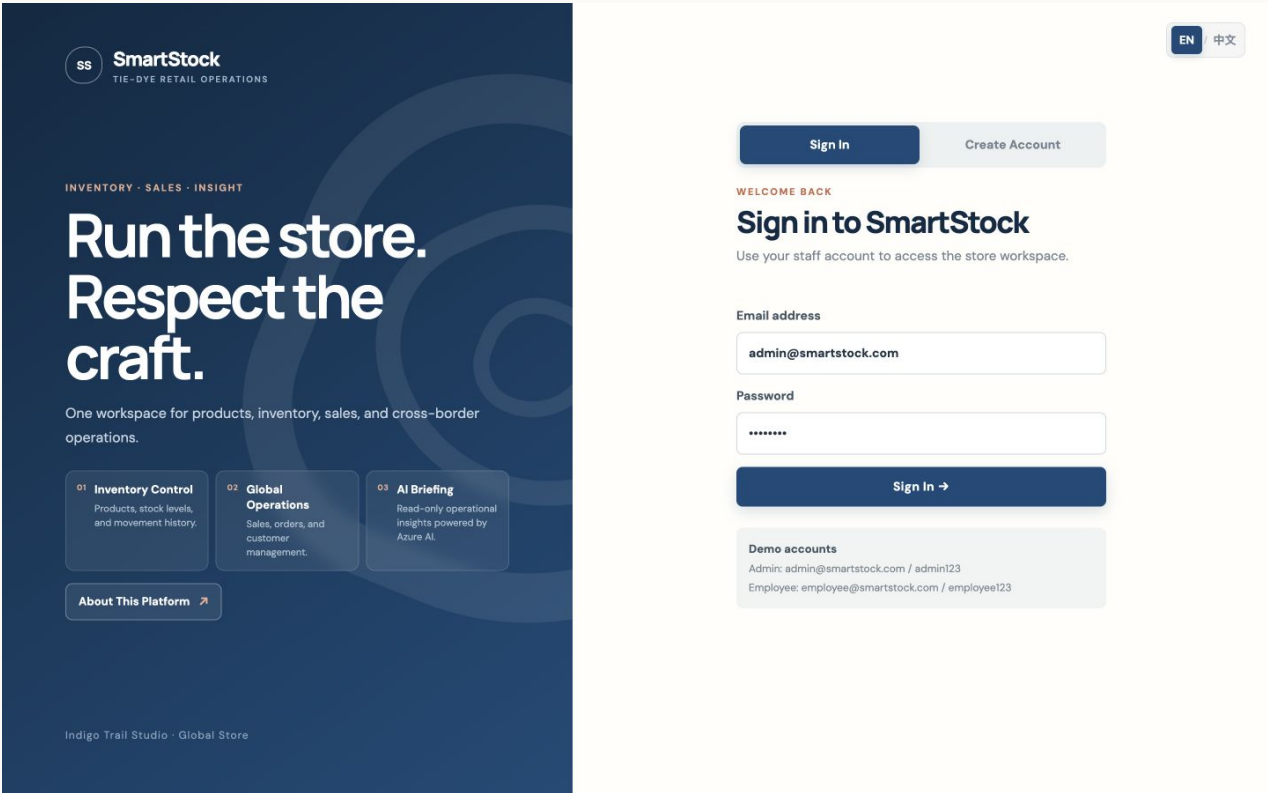
2. Access Information

Application: <https://dyedwild-smartstock.azurewebsites.net>

Demonstration accounts:

Role	Email	Password	Primary access
Administrator	admin@smartstock.com	admin123	All business functions, Database Viewer, AI briefing
Employee	employee@smartstock.com	employee123	Read access and permitted operational actions

These are public demonstration credentials. They must be replaced before any real business deployment.



SmartStock login page

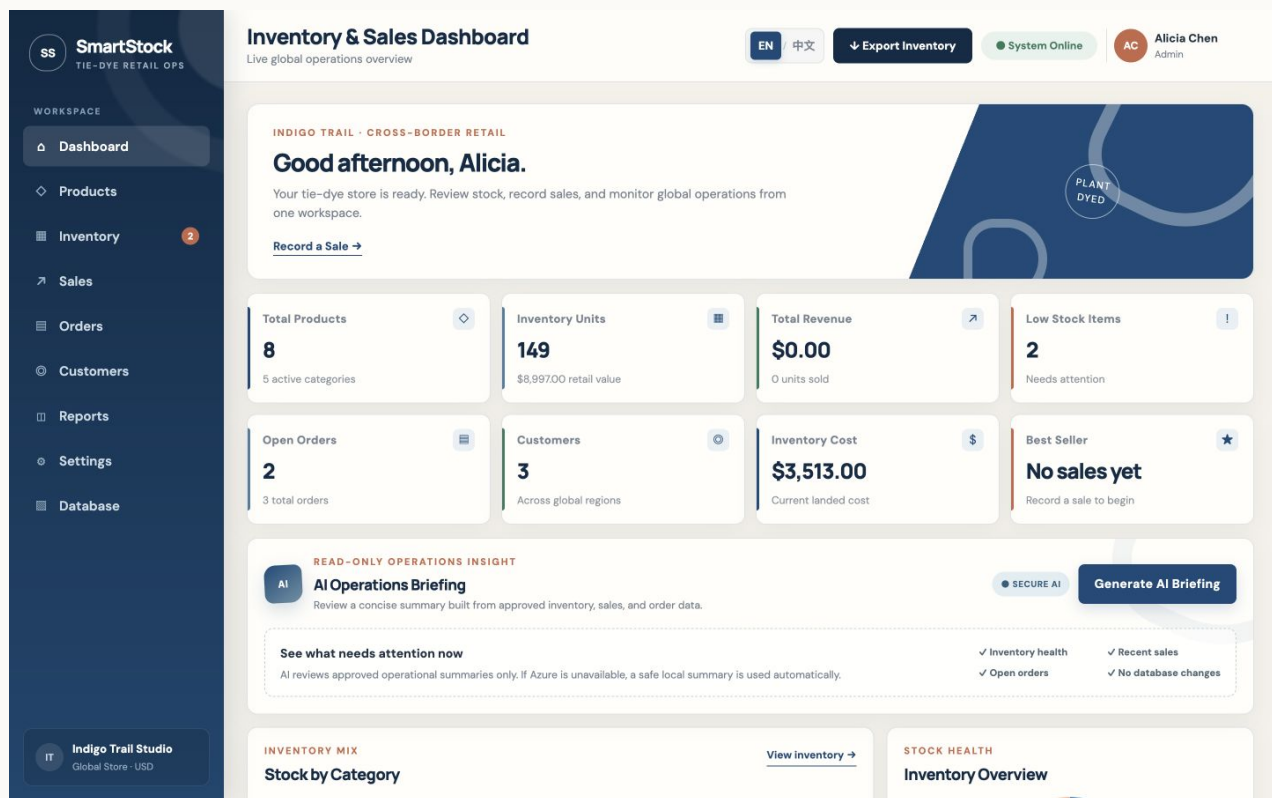
3. Sign In and Language

- 1 Open the application URL.
- 2 Select **EN** or **Chinese** in the upper-right language switch.
- 3 Enter the staff email and password.
- 4 Select **Sign In**.
- 5 Confirm the workspace displays the signed-in name and role.

New users may select **Create Account** to create an employee account. Administrator accounts can be created only by an authenticated administrator.

4. Dashboard

The Dashboard summarizes products, inventory units, retail value, revenue, low-stock items, orders, customers, inventory cost, and best-selling product. Low-stock cards and recent orders identify the next operational actions.

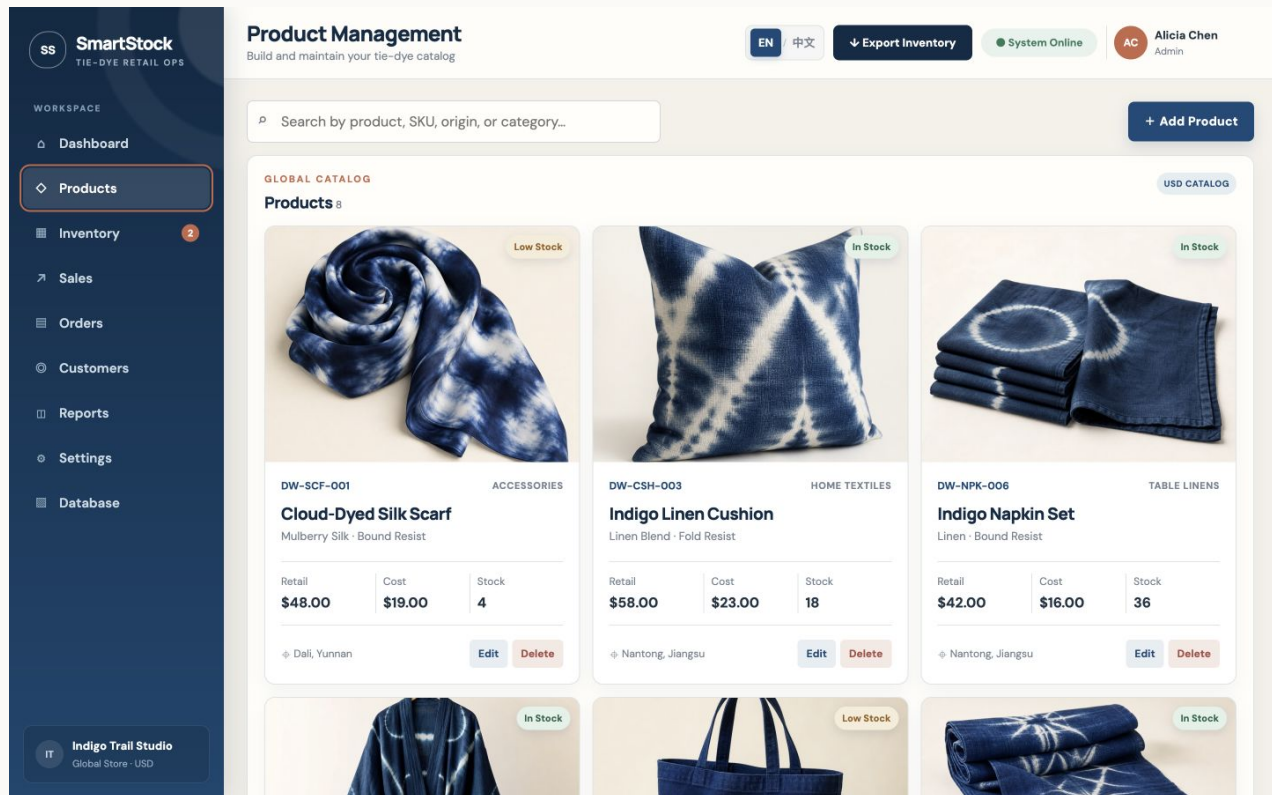


SmartStock dashboard

Use **Export Inventory** to download the current inventory view. Generated files may contain business data and should be handled appropriately.

5. Products

The Products page stores the catalog used by inventory and sales.



SmartStock products page

Add a product

- 1 Open **Products**.
- 2 Select **Add Product**.
- 3 Enter a unique SKU, product name, category, craft origin, material, and dye technique.
- 4 Enter retail price, cost, quantity, and low-stock threshold.
- 5 Choose one image method:
 - paste a public Product Image URL; or
 - select **Choose Image** for a JPG, PNG, or WebP file up to 5 MB.
- 1 Review the image preview.
- 2 Save the product.

Only administrators can create, edit, delete, or upload product images. Products with sales history cannot be deleted.

6. Inventory Movement History

The Inventory page provides an immutable history of quantity changes.

SmartStock inventory movement history

Each movement records:

- product and SKU;
- movement type;
- signed quantity change;
- quantity before and after;
- operator;
- reason;
- source reference;
- timestamp.

Make an administrator adjustment

- 1 Open **Inventory**.
- 2 Select the adjustment action.
- 3 Choose a product.
- 4 Enter a positive or negative change.
- 5 Enter a clear business reason.
- 6 Confirm the adjustment.
- 7 Verify both the product balance and the new movement row.

SmartStock blocks changes that would reduce inventory below zero.

7. Sales

Record a sale

- 1 Open **Sales** or select **Record a Sale** from Dashboard.
- 2 Choose the product.
- 3 Enter quantity, destination region, and sales channel.
- 4 Submit the sale.
- 5 Confirm the sale appears in history.
- 6 Open Inventory and confirm stock decreased with a **SALE** movement.

The server calculates the total from the saved product price. Sale creation and stock reduction occur in one database transaction.

8. Orders

The Orders Sales page tracks purchase and international orders.

Create an order

- 1 Open **Orders**.
- 2 Select **Create Order**.
- 3 Choose the order type and status.
- 4 Enter supplier or channel, product, destination region, and quantity.
- 5 Select **Create Order**.

Administrators can update order status as the order moves from pending to transit, delivery, or cancellation.

9. Customers

The Customer Directory stores cross-border customer contact and region information.

Add a customer

- 1 Open **Customers**.
- 2 Select **Add Customer**.
- 3 Enter name, email, phone, and region.
- 4 Save the record.

Administrators may delete a customer only when the customer has no purchase history. This protects historical business records.

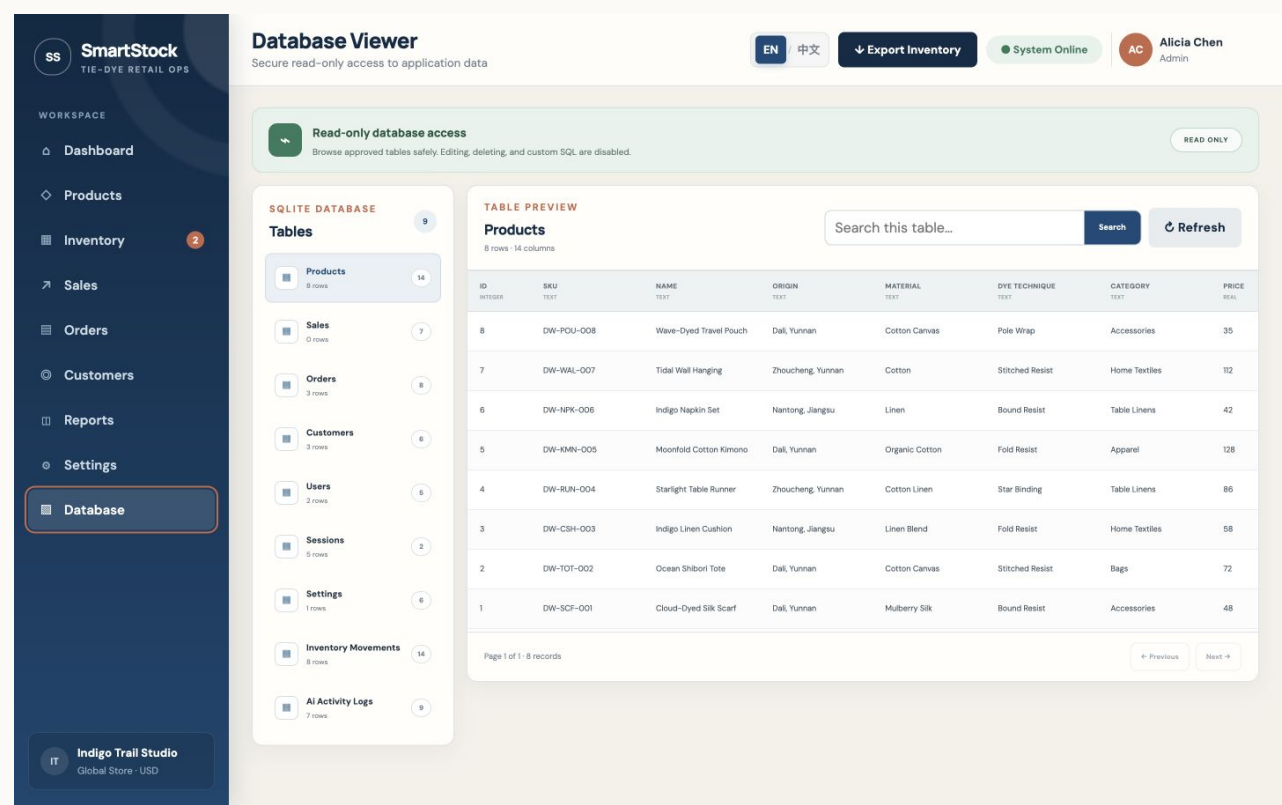
10. Reports and Settings

- **Reports** presents inventory and sales summaries.
- **Settings** stores store name, email, default stock threshold, administrator display name, and currency.

Only administrators can modify settings.

11. Database Viewer

The Database Viewer gives administrators safe, read-only access to approved application tables.



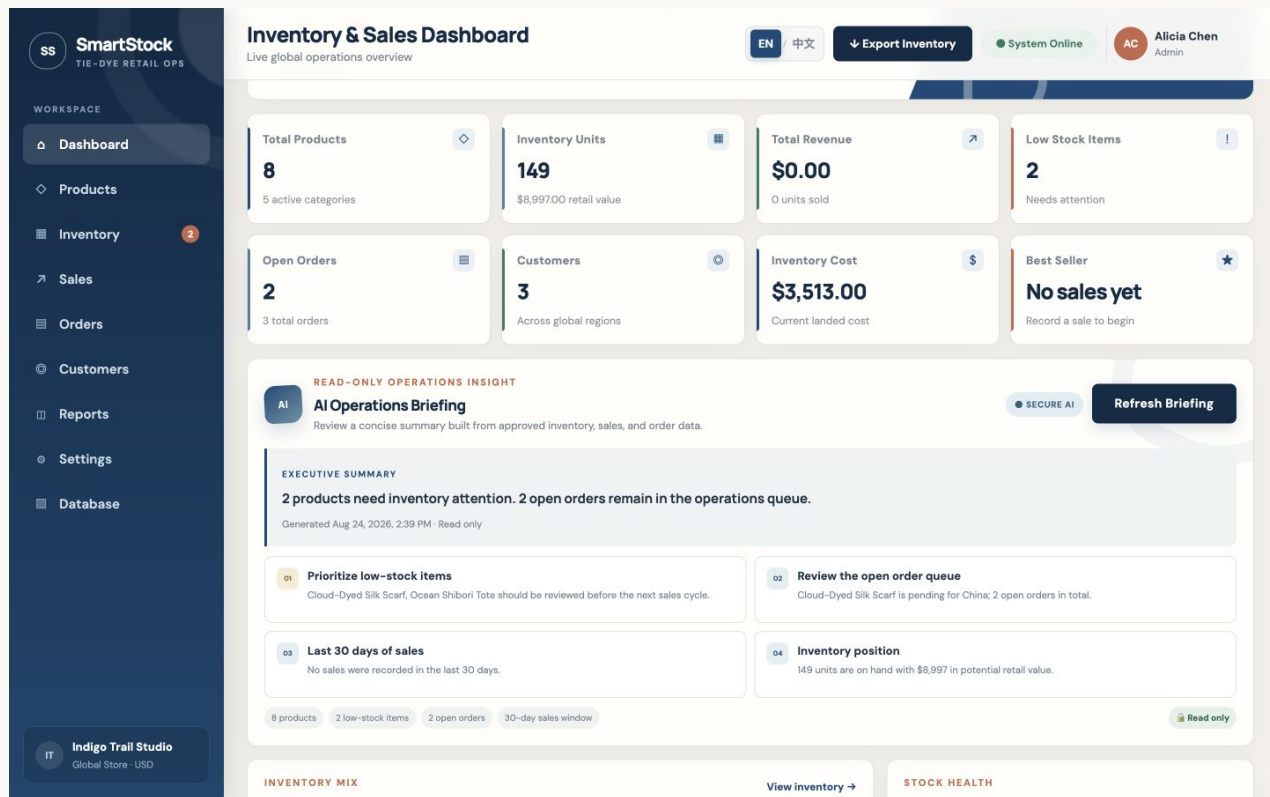
SmartStock read-only Database Viewer

- 1 Open **Database**.
- 2 Select an approved table.
- 3 Search records if needed.
- 4 Use pagination for larger tables.
- 5 Select Refresh to retrieve current records.

Custom SQL, editing, and deletion are disabled. Sensitive password hashes and full session tokens are not displayed.

12. AI Operations Briefing

The administrator-only AI briefing summarizes approved inventory, recent sales, and open-order aggregates. It never changes inventory or creates orders.



SmartStock AI operations briefing

- 1 Open **Dashboard** as an administrator.
- 2 Select **Generate AI Briefing**.
- 3 Review the executive summary and four insights.
- 4 Confirm the card shows **Read only**.
- 5 Select **Refresh Briefing** when current data changes.

In Azure, the card displays **AZURE AI** when the model responds. If Azure AI is temporarily unavailable, SmartStock automatically returns a safe local fallback.

13. Sign Out

- 1 Select the user profile in the upper-right corner.
- 2 Select the sign-out action.
- 3 Confirm the login screen returns.

The previous session token is invalid after logout.

14. Known Limitations and Gotchas

- SmartStock is an internal operations platform; it does not provide consumer checkout.
- The Azure Free F1 plan can cold-start after inactivity.
- SQLite is configured for one App Service instance and demonstration-scale workload.
- The AI briefing is advisory and read-only.
- Image files must be JPG, PNG, or WebP and at most 5 MB.
- Product SKUs must be unique.
- Products and customers linked to historical activity may be protected from deletion.
- Demo credentials are not appropriate for real customer or production data.

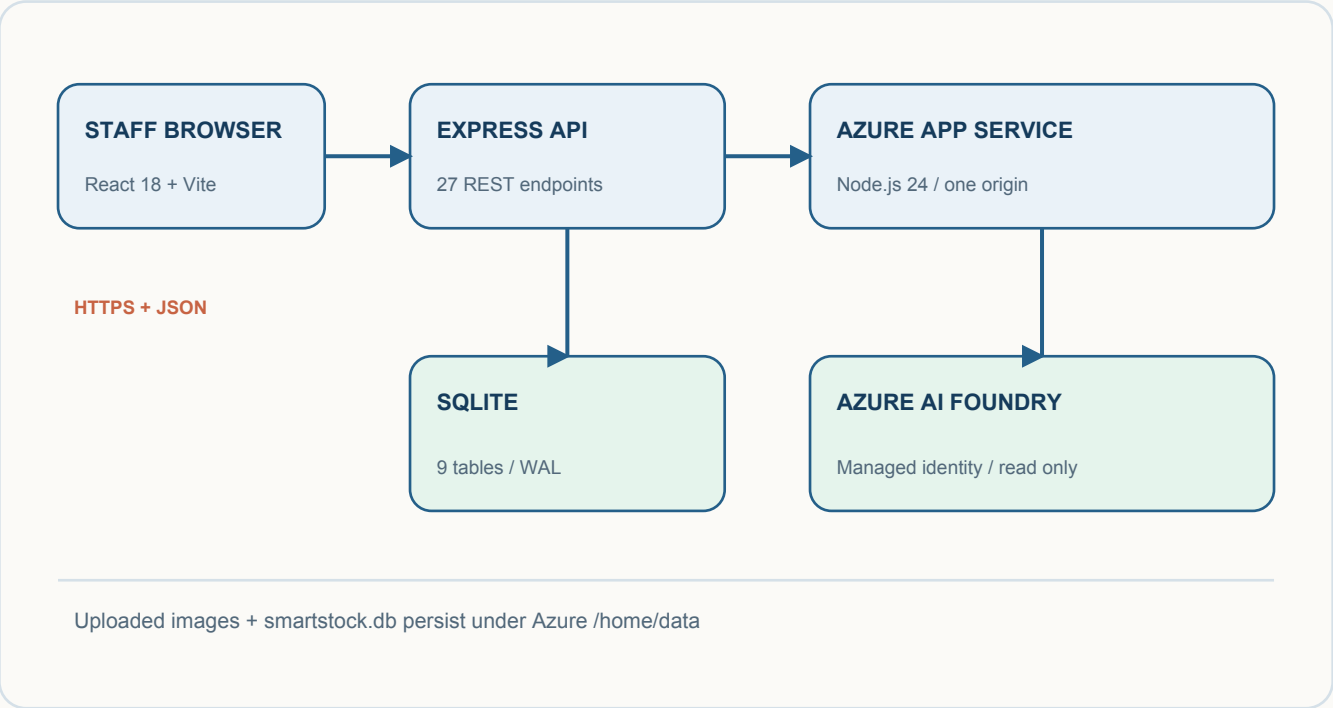
15. Support Contact

For application defects or documentation corrections, open a GitHub issue at <https://github.com/RuhangLiu/SmartStock/issues>. Include the page, time, expected behavior, actual behavior, and a screenshot without passwords or tokens.

System Architecture

1. Architecture Overview

SmartStock is a single Azure-hosted full-stack application. React and Express share one App Service origin. Express owns all database access, authentication, authorization, validation, image persistence, and Azure AI communication.



SmartStock system architecture

2. Component Responsibilities

Component	Technology	Responsibility
Browser client	Current web browser	Renders UI, submits HTTPS requests, stores the active bearer token in application memory
Frontend	React 18 + Vite 5	Staff workflows, bilingual labels, validation feedback, charts, forms, and read-only views
Backend/API	Node.js 24 + Express 4	27 REST endpoints, authentication, role checks, input validation, transactions, static file delivery
Database	Node DatabaseSync + SQLite	Nine application tables, foreign keys, WAL, inventory and sales records
Image storage	Azure App Service /home/data	Uploaded product JPG, PNG, and WebP files
AI service	Azure AI Foundry	Generates four read-only operations insights from allowlisted aggregates
Identity	Azure managed identity	Obtains an Azure AI access token without storing an API key
Source and release	GitHub + Azure Deployment Center	Branch review, main-branch deployment, build and release history

3. Request Flow

3.1 Browser workflow

```
User action
-> React event handler
-> HTTPS request to /api/...
-> Express authentication and role middleware
-> Controller validation and business logic
-> SQLite transaction or query
-> JSON response
-> React state update and user feedback
```

3.2 Postman workflow

```
Postman
-> HTTPS request directly to Express /api endpoint
-> authentication and validation
-> SQLite
-> JSON response directly to Postman
```

Postman does **not** send the request through the React web UI. React and Postman are separate API clients.

3.3 AI workflow

```
Administrator
-> POST /api/ai/briefing
-> Express verifies admin role
-> allowlisted aggregate SQLite queries
-> App Service managed identity token
-> Azure AI model deployment
-> strict JSON parsing and output limits
-> ai_activity_logs audit row
-> four read-only insights
```

If Azure AI fails, the controller creates a deterministic local briefing from the same aggregate object and records the fallback provider.

4. API Layer

API group	Endpoint count	Access summary
Authentication	5	Login/signup public; profile/logout authenticated; staff registration admin
Products	6	Read authenticated; mutations and image upload admin
Sales	3	Authenticated operations
Orders	3	Read authenticated; create/update admin
Customers	3	Read authenticated; create/delete admin
Settings	2	Read authenticated; update admin
Inventory	2	History authenticated; manual adjustment admin
Database	2	Admin-only read-only viewer
AI	1	Admin-only read-only briefing
Total	27	Bearer-token authentication and role-based access

The exact route list is in [API Endpoint Inventory](appendices/API_Endpoint_Inventory.md).

5. Data Layer

Table	Primary purpose
products	Catalog, price, cost, stock, threshold, image
sales	Completed sales and server-calculated totals
users	Staff identity, password hash, role
sessions	Expiring bearer-token sessions
orders	Purchase and international-order tracking
customers	Customer directory and region
settings	Single store configuration row
inventory_movements	Immutable stock change history
ai_activity_logs	Safe AI provider and performance audit

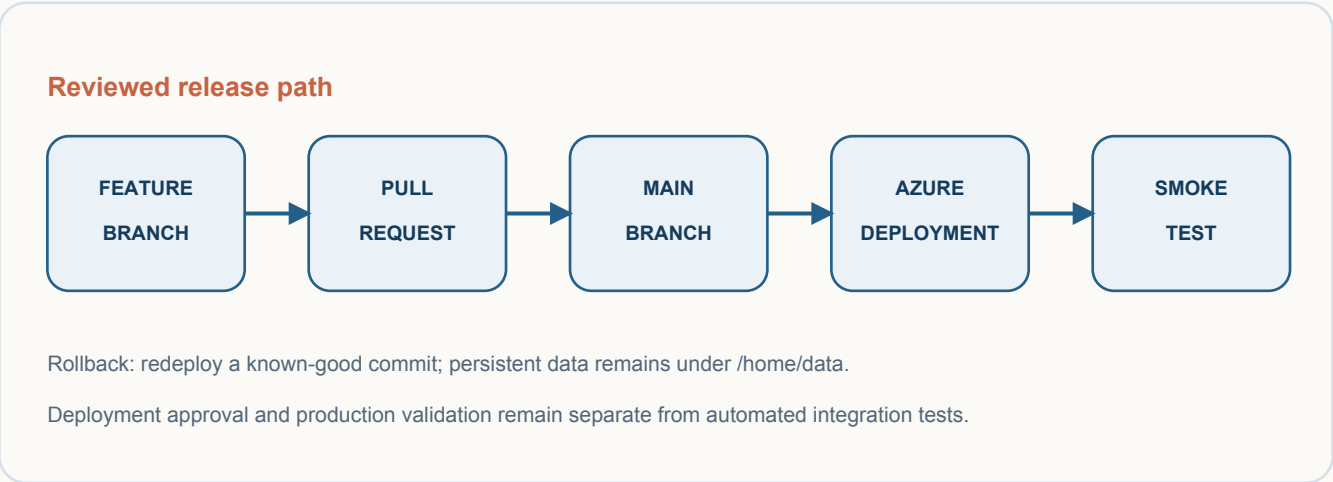
The detailed entity relationships are available in [Database Design](Database_Design.pdf).

6. Hosting Environments

Environment	Frontend/API	Database and files	AI mode
Automated test	Ephemeral local HTTP server	Temporary SQLite and upload folder	Local preview
Local development	Vite + Express or combined Express build	Repository-local SQLite and uploads	Local preview by default
Production	Azure App Service, one origin	/home/data persistent storage	Azure AI with local fallback

SmartStock currently has no separate staging environment. Reviewed pull requests and automated tests reduce risk before main-branch deployment; a staging slot is a future improvement.

7. Deployment Architecture



SmartStock deployment pipeline

The release path is feature branch -> automated tests -> pull request -> main -> Azure build -> App Service -> smoke test. Application rollback does not replace the persistent database.

8. Security Boundaries

- Passwords use salted script hashes.
- Sessions expire after seven days and are deleted on logout.
- Every protected business route requires a bearer token.
- Administrative mutations use role middleware.
- SQL uses prepared statements and server-defined queries.
- The Database Viewer uses an allowlist and removes sensitive fields.
- Image upload validates role, MIME type, file count, size, and server-generated filename.
- Azure AI receives approved aggregate data only.
- Managed identity avoids an Azure AI key in React, GitHub, or App Service settings.
- AI output is parsed into a strict structure and cannot execute SQL.

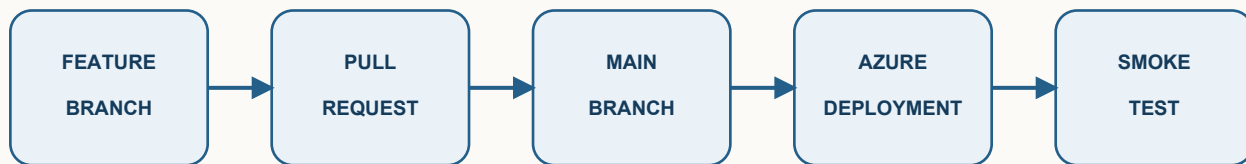
9. Architecture Constraints

- SQLite file access is synchronous and appropriate for a small single-instance capstone workload.
- Multi-instance horizontal scaling would require migration to a managed network database.
- Azure Free F1 is a demonstration plan and may cold-start.
- No external customer checkout, payment, shipping, or tax service is connected.
- Monitoring is based on logs, smoke tests, and audit tables rather than a dedicated observability platform.

Deployment Pipeline and Security Considerations

1. Deployment Pipeline

Reviewed release path



Rollback: redeploy a known-good commit; persistent data remains under /home/data.

Deployment approval and production validation remain separate from automated integration tests.

SmartStock deployment pipeline

- 1 Create a scoped feature branch.
- 2 Implement code and documentation changes.
- 3 Run the 45-case API suite against temporary SQLite.
- 4 Open a pull request and review the diff and test evidence.
- 5 Merge to **main** after approval.
- 6 Azure Deployment Center builds the Vite client and starts the Node.js application.
- 7 Run read-only production smoke tests.

Microsoft documents GitHub-based App Service deployment through Deployment Center in [Deploy to App Service using GitHub Actions](#).

2. Approval and Rollback

- Pull-request approval is the release gate.
- Automated tests must pass before merge.
- A deployment is not complete until the production smoke test passes.
- If the application fails, redeploy the last known-good commit or revert through a new pull request.
- Do not roll back the SQLite file automatically with the application package.

3. Authentication and Authorization

- Passwords are salted and hashed with `bcrypt`.

- Bearer tokens are stored in `sessions` and expire after seven days.
- `requireAuth` protects business endpoints.
- `requireRole('admin')` protects sensitive operations.
- Public signup creates employee accounts only.

4. Database Security

- Prepared statements are used for application values.
- Foreign keys and transactions protect consistency.
- Sale creation and inventory reduction are atomic.
- Inventory changes and movement-history creation are atomic.
- The admin Database Viewer uses approved table definitions and does not accept arbitrary SQL.
- Password hashes and complete session tokens are excluded from viewer results.

5. File Upload Security

- Administrator-only endpoint.
- One file per request.
- Allowed MIME types: JPEG, PNG, WebP.
- Maximum size: 5 MB.
- Server-generated UUID filenames.
- Persistent storage outside the deployable application package.

6. Azure AI Security

- The browser never calls Azure AI directly.
- The server reads only allowlisted inventory, order, and sales aggregates.
- Detailed lists are limited to five rows.
- Output is limited to four insights and parsed as strict JSON.
- The AI cannot execute SQL or call mutation endpoints.
- Every request writes a safe activity log without prompt text, password, bearer token, or Azure token.
- App Service uses a system-assigned managed identity with the least-privilege `Cognitive Services OpenAI User` role.
- A deterministic local fallback keeps the dashboard usable when Azure AI is unavailable.

7. Secret Management

Allowed nonsecret application settings include endpoints, paths, deployment names, and feature flags. The following must never be committed:

- Azure access tokens or API keys;
- real passwords;
- bearer/session tokens;
- production database copies;
- private customer data;
- exported logs containing authorization headers.

8. Production Hardening Recommendations

The capstone is safe for demonstration, but a commercial deployment should also:

- 1 Replace public demo credentials and enforce strong password policy.
- 2 Add HTTPS security headers, request-rate limits, and CSRF review.
- 3 Add centralized telemetry, alerts, and a protected health endpoint.
- 4 Add scheduled database backups and tested recovery procedures.
- 5 Migrate to a managed database before multi-instance scaling.
- 6 Add a staging environment and protected deployment approvals.
- 7 Add dependency and secret scanning to CI.

API Endpoint Inventory

All business endpoints return JSON and use the `/api` prefix. Except for login and signup, requests require **Authorization: Bearer <token>**. **Admin** means an authenticated administrator; **Staff** means either administrator or employee.

#	Method	Endpoint	Access	Purpose
1	POST	<code>/api/auth/login</code>	Public	Authenticate staff and create a session
2	POST	<code>/api/auth/signup</code>	Public	Create an employee account and session
3	GET	<code>/api/auth/me</code>	Staff	Return current user profile
4	POST	<code>/api/auth/logout</code>	Staff	Invalidate current session
5	POST	<code>/api/auth/register</code>	Admin	Create another staff account
6	GET	<code>/api/products</code>	Staff	List and search products
7	GET	<code>/api/products/low-stock</code>	Staff	List products at or below threshold
8	POST	<code>/api/products/image</code>	Admin	Upload a validated product image
9	POST	<code>/api/products</code>	Admin	Create product
10	PUT	<code>/api/products/:id</code>	Admin	Update product
11	DELETE	<code>/api/products/:id</code>	Admin	Delete product without sales history
12	GET	<code>/api/sales</code>	Staff	List sales
13	POST	<code>/api/sales</code>	Staff	Record sale and reduce inventory
14	GET	<code>/api/sales/report</code>	Staff	Return dashboard aggregates
15	GET	<code>/api/orders</code>	Staff	List orders
16	POST	<code>/api/orders</code>	Admin	Create order
17	PUT	<code>/api/orders/:id</code>	Admin	Update order status
18	GET	<code>/api/customers</code>	Staff	List customers
19	POST	<code>/api/customers</code>	Admin	Create customer
20	DELETE	<code>/api/customers/:id</code>	Admin	Delete customer without purchase history
21	GET	<code>/api/settings</code>	Staff	Read store settings
22	PUT	<code>/api/settings</code>	Admin	Update store settings
23	GET	<code>/api/inventory/movements</code>	Staff	Search and paginate movement history
24	POST	<code>/api/inventory/adjustments</code>	Admin	Apply documented stock adjustment
25	GET	<code>/api/database/tables</code>	Admin	List approved database tables
26	GET	<code>/api/database/:table</code>	Admin	Search and paginate safe table rows
27	POST	<code>/api/ai/briefing</code>	Admin	Generate bilingual read-only operations briefing

Endpoint Count Reconciliation

Postman collection counts and automated test counts measure different things:

- **27 endpoints** are distinct method-and-path API operations.
- **45 test cases** cover those endpoints under multiple conditions, including success, invalid input, insufficient stock, forbidden roles, expired sessions, safe field filtering, file validation, and database change verification.

Test and Smoke Evidence

Automated Test Summary

Metric	Result
Executed	45
Passed	45
Failed	0
Pass rate	100%
Runtime	Node.js 24
Database	Isolated temporary SQLite database
Transport	Real HTTP on an ephemeral local port

Machine-readable report: `[.../evidence/API_Test_Results.json](.../evidence/API_Test_Results.json)`

Spreadsheet report: `[.../API_Test_Cases_and_Results.xlsx](.../API_Test_Cases_and_Results.xlsx)`

Coverage Categories

- Authentication and session invalidation
- Role-based access control
- Product CRUD and duplicate SKU validation
- Image upload permissions, type validation, and persistence
- Sales transaction and stock reduction
- Insufficient-stock rejection
- Order and customer operations
- Protected deletion rules
- Settings access
- Read-only Database Viewer and safe-field filtering
- Inventory movement history and manual adjustments
- Negative-stock protection
- AI role restriction, bilingual preview, and safe audit logging

Production Smoke Evidence

Date: **August 24, 2026** URL: **<https://dyedwild-smartstock.azurewebsites.net>**

Step	Result
Load production root over HTTPS	Pass
Sign in as administrator	Pass
Load Dashboard aggregates	Pass
Open Database Viewer	Pass
Confirm approved SQLite tables	Pass
Generate Azure AI briefing	Pass; AZURE AI and four read-only insights displayed
Confirm no business mutation during smoke test	Pass

Evidence Handling

Screenshots in **docs/screenshots** use demonstration data. No password field contents, bearer tokens, Azure access tokens, or customer-sensitive production records are included.

Appendix C - Artifact Index

The combined PDF is supported by the following reviewable repository artifacts. The spreadsheet and JSON evidence are generated from the final automated test run rather than manually entered status claims.

Artifact	Purpose
docs/API_Test_Cases_and_Results.xlsx	Three-sheet test report with 45 cases and 27-endpoint traceability
evidence/API_Test_Results.json	Machine-readable final test execution report
server/tests/api-test-runner.js	Automated HTTP integration test runner
docs/SmartStock_API.postman_collection.json	Saved API requests for manual demonstration
docs/screenshots/	Current user interface evidence
docs/diagrams/	Repository-native architecture and release diagrams
server/database/schema.sql	SQLite schema source
README.md	Repository entry point and start commands

Final Evidence Statement

On August 24, 2026, the final automated suite executed 45 test cases against the real Express application using an isolated temporary SQLite database. All 45 tests passed. The endpoint coverage sheet maps every one of the 27 finalized REST endpoints to at least one passing case. A separate production smoke test confirmed HTTPS load, administrator login, dashboard data, the read-only Database Viewer, and a live Azure AI briefing without changing business records.